RL - ÇALIŞMA 1 : Continuous-Control CartPole Üzerinde PPO ile Reinforcement Learning Kavramlarının Deneysel İncelenmesi

Osman Melih Muslu

15 Ekim 2026

Özet

Bu raporda continuous-control CartPole sistemi üzerinde Proximal Policy Optimization (PPO) kullanarak temel Reinforcement Learning (RL) kavramlarını deneysel olarak incelemeye çalıştım. Özellikle David Silver'ın Reinforcement Learning ders serisinin ilk beş Lecture'ında ele aldığı State, Policy, Reward, Markov Decision Process (MDP), Discounted Return, Value Function, Model-Free Prediction ve Model-Free Control kavramlarını CartPole örneği üzerinden anlamlandırmayı amaçladım. Fiziksel plant'in kontrol girdisine nasıl tepki verdiğini açıklamak amacıyla CartPole'un ideal nonlinear hareket denklemlerini Newton–Euler yöntemiyle de türettim.

Learning algorithm olarak `safe-control-gym` (https://github.com/learnsyslab/safe-control-gym) içerisinde bulunan hazır PPO implementation'ı kullandım. PPO algoritmasını sıfırdan yazmak yerine, hazır implementation üzerinden CartPole problemindeki temel RL kavramlarını incelemeyi hedefledim.

`safe-control-gym` içerisindeki CartPole örneğinde temel training/evaluation akışı ve iki adet response grafiği hazır olarak bulunuyordu. Ben bu yapıyı deney yapabileceğim daha kapsamlı bir RL labına çevirmek için genişlettim.

Deneysel analiz için Reward weight'lerini, Discount Factor γ, GAE parametresi λ ve PPO clipping parameter ϵ gibi bazı RL parametrelerini değiştirilebilir hale getirdim. Ayrıca Observation Noise, External Disturbance ve Action Delay gibi robustness testleri ekledim.

GUI'ye CartPole sisteminin real-time 2D animasyonunu da ekledim. Böylece controller'ın ürettiği Action'ın cart ve pole üzerindeki etkisini yalnızca grafiklerden değil, sistem hareketi üzerinden de anlık olarak gözlemleyebildim.

Her Evaluation sonunda Final Error, RMSE, RMS Force, maximum Pole Angle, maximum Force, cumulative Reward ve Settling Time gibi performance metrikleri otomatik olarak hesaplanıyor.

Beş temel deney yaptım: farklı Initial State'lere karşı generalization, position Reward weight q_x değişimi, Control Effort penalty R_u değişimi, Discount Factor γ değişimi ve Observation Noise altında robustness analizi.

Sonuçlarda öğrenilen Policy'nin sistemin anlık State'ine göre farklı Action ürettiğini, Reward Design'ın öğrenilen kontrol trade-off'unu doğrudan değiştirdiğini, Discount Factor'ın short-term ve long-term performans arasındaki dengeyi etkilediğini ve nominal koşullarda iyi çalışan bir Policy'nin noisy measurement durumunda belirgin biçimde daha fazla Control Activity üretebildiğini gözlemledim. Uygun yerlerde LQR ile kavramsal bağlantılar da kurdum.

1 Giriş

Bu çalışmada PPO algoritmasını sıfırdan implement etmek yerine çalışan bir PPO implementation kullanarak RL kavramlarının continuous CartPole sistemi üzerinde nasıl karşılık bulduğunu deneysel olarak incelemek istedim.

CartPole State'ini

$$\boldsymbol{s}_t = \begin{bmatrix} x_t & \dot{x}_t & \theta_t & \dot{\theta}_t \end{bmatrix}^T \tag{1}$$

şeklinde kullandım. Burada x cart position, $\dot{x}$ cart velocity, θ pole angle ve $\dot{\theta}$ pole angular velocity'dir.

Stabilization hedefini

$$\boldsymbol{s}_{\text{ref}} = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T \tag{2}$$

olarak belirledim.

PPO Actor, gözlenen State'ten continuous Action üretir:

$$a_t \sim \pi_\phi(a \mid \boldsymbol{s}_t). \tag{3}$$

Bu Action daha sonra fiziksel cart force F_t değerine ölçeklenir ve Environment bir sonraki State ile Reward'u üretir:

$$(\boldsymbol{s}_t, a_t) \longrightarrow (\boldsymbol{s}_{t+1}, r_{t+1}). \tag{4}$$

Bu raporda yalnızca "PPO CartPole'u stabilize edebiliyor mu?" sorusuna bakmak istemedim. Asıl olarak RL tasarım parametrelerini değiştirdiğimde öğrenilen Policy davranışının nasıl değiştiğini gözlemlemeye çalıştım.

PPO Agent'ın etkileşime girdiği plant'i daha açık tanımlamak için önce Bölüm 2'te Newton–Euler yöntemiyle CartPole'un ideal nonlinear hareket denklemlerini türettim. Ardından bu dinamikleri temel alan RL deney platformunu ve yaptığım karşılaştırmaları sundum.

2 CartPole Fiziksel Modeli ve Newton–Euler Yöntemiyle Türetilmesi

PPO Agent'ın ürettiği Action'ın fiziksel sistem üzerinde neyi değiştirdiğini anlamak amacıyla CartPole'un nonlinear hareket denklemlerini Newton'un ikinci yasası ve moment dengesi üzerinden türettim. Bu bölümde amaç, kontrol girdisi olan yatay kuvvet $F(t)$ ile cart position $x(t)$ ve pole angle $\theta(t)$ arasındaki dinamik ilişkiyi kurmaktır. Başlangıçta cart/pivot sürtünmeleri ve diğer ideal olmayan etkiler ihmal edilmektedir. Dolayısıyla aşağıdaki model, kullanılan simülatörün bütün ayrıntılarını birebir temsil ettiği doğrulanmış bir model değil, CartPole mekaniğinin idealize edilmiş açıklamasıdır.

2.1 System Definition

Cart yatay eksende hareket etmekte; pole ise cart üzerindeki bir pivot etrafında dönmektedir. Pole angle θ, yukarı yönlü düşey eksenden ölçülmekte ve sağa doğru eğilme pozitif kabul edilmektedir. Buna göre $\theta = 0$ dik (unstable) equilibrium point'i temsil eder. Modelde kullanılan parametreler:

M : Cart mass [kg],

m : Pole mass [kg],

l : Pivot–pole center of mass distance [m],

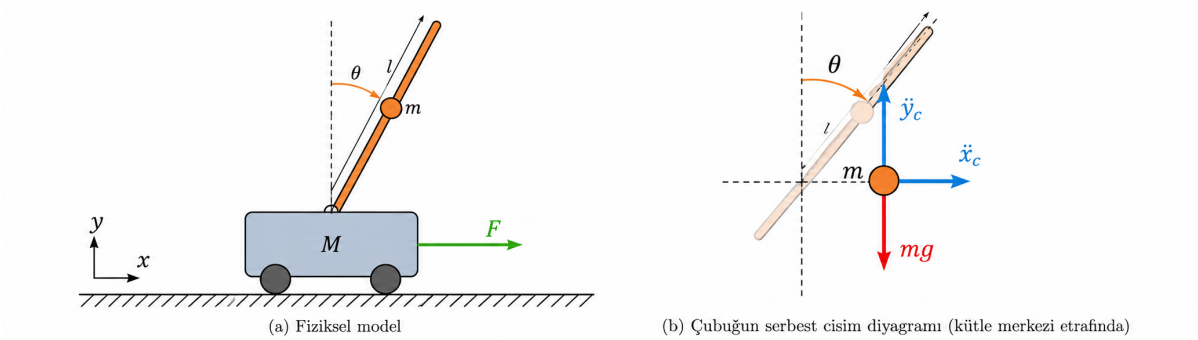
I : Pole moment of inertia about its center of mass [kg m^2],

g : Gravitational acceleration [m/s^2],

F : Cart'a uygulanan horizontal control force [N].

Sistemin iki generalized coordinate'ı x ve θ'dır. Daha önce Denklem (1) ile tanımlanan dört bileşenli State ise bu iki koordinat ve bunların hızlarından oluşmaktadır:

$$\boldsymbol{s}(t) = \begin{bmatrix} x(t) & \dot{x}(t) & \theta(t) & \dot{\theta}(t) \end{bmatrix}^T. \tag{5}$$



(a) Fiziksel model

(b) Çubuğun serbest cisim diyagramı (kütle merkezi etrafında)

Şekil 1: CartPole sistemi. (a) Araba (kütle M) ve mafsallı çubuk (kütle m) ile birlikte fiziksel model. Çubuğun kütle merkezi pivottan l uzaklıktadır ve açı $\theta = 0$ dik konumu temsil eder. Kontrol girdisi F arabanın yatay yönde uyguladığı kuvvettir. (b) Çubuğun kütle merkezi etrafındaki serbest cisim diyagramı. Kütle merkezinin ivme bileşenleri $\ddot{x}_c$ ve $\ddot{y}_c$ gösterilmiştir.

Şekil 1: CartPole sisteminin fiziksel modeli ve çubuğun kütle merkezi etrafındaki serbest cisim diyagramı. Burada M araba kütlesini, m çubuk kütlesini, l pivot ile kütle merkezi arasındaki mesafeyi, θ çubuk açısını ve F kontrol kuvvetini göstermektedir.

2.2 Pole Center of Mass Kinematics

Pivot cart üzerinde bulunduğundan pole center of mass konumunu

$$x_c = x + l\sin\theta, \tag{6}$$

$$y_c = l\cos\theta \tag{7}$$

şeklinde yazabiliriz. Zamana göre türevleri:

$$\dot{x}_c = \dot{x} + l\dot{\theta}\cos\theta, \tag{8}$$

$$\dot{y}_c = -l\dot{\theta}\sin\theta. \tag{9}$$

Bir kez daha türev alındığında center of mass ivmeleri elde edilir:

$$\boxed{\ddot{x}_c = \ddot{x} + l\ddot{\theta}\cos\theta - l\dot{\theta}^2\sin\theta}, \tag{10}$$

$$\boxed{\ddot{y}_c = -l\ddot{\theta}\sin\theta - l\dot{\theta}^2\cos\theta}. \tag{11}$$

2.3 Horizontal Force Balance: Newton'un İkinci Yasası

Pivot'un pole'a uyguladığı yatay kuvveti H_x ile gösterelim. Pole için Newton'un ikinci yasası:

$$H_x = m\ddot{x}_c. \tag{12}$$

Etki–tepki prensibi gereği pivot, cart üzerine ters yönde $-H_x$ kuvveti uygular. Bu nedenle cart için yatay kuvvet dengesi:

$$F - H_x = M\ddot{x}. \tag{13}$$

Denklem (12) ve (13) birleştirildiğinde:

$$F = M\ddot{x} + m\ddot{x}_c. \tag{14}$$

Denklem (10) yerine yazılırsa:

$$F = M\ddot{x} + m\left(\ddot{x} + l\ddot{\theta}\cos\theta - l\dot{\theta}^2\sin\theta\right) \tag{15}$$

$$= (M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta. \tag{16}$$

Böylece cart için birinci nonlinear differential equation elde edilir:

$$\boxed{(M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F} \tag{17}$$

Buradaki $ml\ddot{\theta}\cos\theta$ terimi cart ve pole hareketleri arasındaki dynamic coupling'i; $ml\dot{\theta}^2\sin\theta$ ise açısal hızın karesine bağlı nonlinear etkiyi gösterir.

2.4 Pole Rotational Dynamics: Moment Dengesi

Dönme hareketini pole'un center of mass noktası etrafında inceleyelim. Bu nokta etrafındaki atalet momenti I olduğundan, pozitif θ yönü için moment dengesi $\sum \tau_c = I\ddot{\theta}$ biçimindedir. Pivot'un pole'a uyguladığı düşey kuvveti H_y olarak tanımlayalım. Pole'un düşey hareketi için:

$$H_y - mg = m\ddot{y}_c \quad \Longrightarrow \quad H_y = m\left(g + \ddot{y}_c\right). \tag{18}$$

Pivot kuvvetlerinin kütle merkezi etrafındaki momenti, yukarıda seçilen pozitif açısal yön dikkate alındığında:

$$I\ddot{\theta} = lH_y\sin\theta - lH_x\cos\theta. \tag{19}$$

$H_x = m\ddot{x}_c$ ve $H_y = m(g + \ddot{y}_c)$ ifadelerini yerine yazarsak:

$$I\ddot{\theta} = ml(g + \ddot{y}_c)\sin\theta - ml\ddot{x}_c\cos\theta. \tag{20}$$

Kütle merkezi ivmeleri, Denklem (10) ve (11) kullanılarak:

$$\begin{aligned} I\ddot{\theta} = {} & ml\sin\theta\left(g - l\ddot{\theta}\sin\theta - l\dot{\theta}^2\cos\theta\right) \\ & - ml\cos\theta\left(\ddot{x} + l\ddot{\theta}\cos\theta - l\dot{\theta}^2\sin\theta\right). \end{aligned} \tag{21}$$

Parantezler açıldığında, $\dot{\theta}^2\sin\theta\cos\theta$ ile orantılı iki terim birbirini götürür:

$$\begin{aligned} I\ddot{\theta} = {} & mgl\sin\theta - ml\ddot{x}\cos\theta \\ & - ml^2\ddot{\theta}\underbrace{(\sin^2\theta + \cos^2\theta)}_{=1}. \end{aligned} \tag{22}$$

Sonuç olarak pole için ikinci nonlinear differential equation:

$$\boxed{(I + ml^2)\ddot{\theta} + ml\ddot{x}\cos\theta - mgl\sin\theta = 0} \tag{23}$$

2.5 Coupled Nonlinear Equations of Motion

Newton'un yatay kuvvet dengesi ve pole moment dengesi birlikte:

$$\boxed{\begin{aligned} &(M + m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F, \\ &ml\ddot{x}\cos\theta + (I + ml^2)\ddot{\theta} - mgl\sin\theta = 0. \end{aligned}} \tag{24}$$

Bu iki bağlı, ikinci dereceden nonlinear ODE; matris gösterimiyle:

$$\underbrace{\begin{bmatrix} M + m & ml\cos\theta \\ ml\cos\theta & I + ml^2 \end{bmatrix}}_{\boldsymbol{M}(q)} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} + \underbrace{\begin{bmatrix} -ml\dot{\theta}^2\sin\theta \\ -mgl\sin\theta \end{bmatrix}}_{\boldsymbol{h}(q,\dot{q})} = \underbrace{\begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\boldsymbol{B}} F. \tag{25}$$

2.6 State-Space Representation ile PPO Bağlantısı

PPO Environment, uygulanan $F(t)$ kuvvetine karşılık sistemin State'inin nasıl değiştiğini hesaplar. Bunun için Denklem (24)'daki iki bilinmeyen ivme çözülebilir. Kısaltma olarak:

$$H = I + ml^2, \tag{26}$$

$$p = F + ml\dot{\theta}^2\sin\theta, \tag{27}$$

$$z = mgl\sin\theta, \tag{28}$$

$$D = (M + m)H - m^2l^2\cos^2\theta \tag{29}$$

tanımlanır. O zaman:

$$\boxed{\ddot{x} = \frac{Hp - ml\cos\theta\, z}{D}}, \tag{30}$$

$$\boxed{\ddot{\theta} = \frac{(M+m)z - ml\cos\theta\, p}{D}}. \tag{31}$$

Denklem (1)'teki State sıralamasına göre continuous-time nonlinear state equation:

$$\boxed{\dot{\boldsymbol{s}} = \begin{bmatrix} \dot{x} \\ \ddot{x}(\boldsymbol{s}, F) \\ \dot{\theta} \\ \ddot{\theta}(\boldsymbol{s}, F) \end{bmatrix} = \boldsymbol{f}(\boldsymbol{s}, F)} \tag{32}$$

şeklinde yazılır. Continuous action üreten PPO Agent, bu dinamiklere uygulanan kuvveti State'e bağlı olarak seçer. Bu nedenle PPO Policy ile fiziksel plant'i birbirinden ayırmak mümkündür: Policy F girdisini üretirken plant, aynı dinamik denklemler doğrultusunda State'i günceller.

Homojen ve uzunluğu $2l$ olan bir pole kabul edilirse center of mass etrafındaki atalet momenti $I = \frac{1}{3}ml^2$ olur ve $I + ml^2 = \frac{4}{3}ml^2$ elde edilir.

3 Deneysel Platform

Önceki bölümde elde edilen nonlinear dinamikler, Action'ın fiziksel etkisini açıklamak için teorik bir temel sağlar. Deneylerde ise aşağıda anlatılan `safe-control-gym` Environment'ını kullandım.

3.1 Software ve Environment

Deneylerde `safe-control-gym` framework'ünü kullandım. Bu framework, PyBullet tabanlı Cart-Pole ve quadrotor Environment'ları ile learning-based control ve RL deneylerine yönelik araçlar sunmaktadır (LearnSysLab, *safe-control-gym: PyBullet CartPole and Quadrotor Environments for Learning-Based Control and Reinforcement Learning*. Available: https://github.com/learnsyslab/safe-control-gym).

PPO implementation framework içerisinde hazırdı. Ben bu çalışmada CartPole target ve config ayarlarını değiştirdim, farklı Reward ve PPO parametreleriyle yeniden training yaptım ve training/evaluation işlemlerini ayıran özel bir RL Lab GUI geliştirdim.

GUI'de Training ve Evaluation işlemlerini özellikle ayrı tuttum:

- **Training:** PPO Policy parametrelerini update ettim.
- **Evaluation:** trained Policy'yi freeze edip davranışını test ettim.

Initial State ve Observation Noise deneylerinde aynı trained Policy'yi korudum ve yalnızca evaluation koşullarını değiştirdim. Reward weight ve Discount Factor deneylerinde ise objective değiştiği için yeni Policy'ler train ettim.

3.2 GUI Üzerinde Eklediğim Özellikler

`safe-control-gym` içerisindeki mevcut CartPole örneğinde temel training/evaluation akışı ve sınırlı görselleştirme imkânları bulunuyordu. Ben bu yapıyı deney yapabileceğim daha kapsamlı bir RL laboratuvarına dönüştürmek için genişlettim.

İlk olarak CartPole stabilization hedefini

$$\boldsymbol{s}_{\text{ref}} = [0, 0, 0, 0]^T$$

olarak değiştirdim ve PPO Policy'yi bu yeni hedef için tekrar train ettim.

Daha sonra farklı Initial State değerlerini elle girebildiğim, daha önce train ettiğim Policy'leri seçip evaluate edebildiğim ve farklı RL parametreleriyle yeni Policy'ler train edebildiğim *RL CartPole Lab* isimli bir GUI geliştirdim.

GUI'ye CartPole'un real-time hareketini ekledim. Ayrıca yalnızca cart position ve pole angle response'larını görmek yerine

$$x(t), \qquad \theta(t), \qquad F(t), \qquad r(t)$$

sinyallerini aynı anda takip edebileceğim grafikler ekledim.

PPO'nun ürettiği raw Action'ı, fiziksel olarak uygulanan Force'u, actuator saturation durumunu ve anlık Reward değerini de ekranda gösterdim.

Reward Design'ı deneysel olarak inceleyebilmek için

$$q_x,\ q_{\dot{x}},\ q_\theta,\ q_{\dot{\theta}},\ R_u$$

weight'lerini GUI üzerinden değiştirebilir hale getirdim.

Benzer şekilde Discount Factor γ, GAE parametresi λ, PPO clipping parameter ϵ gibi bazı RL parametrelerini de değiştirebildiğim bir PPO bölümü ekledim.

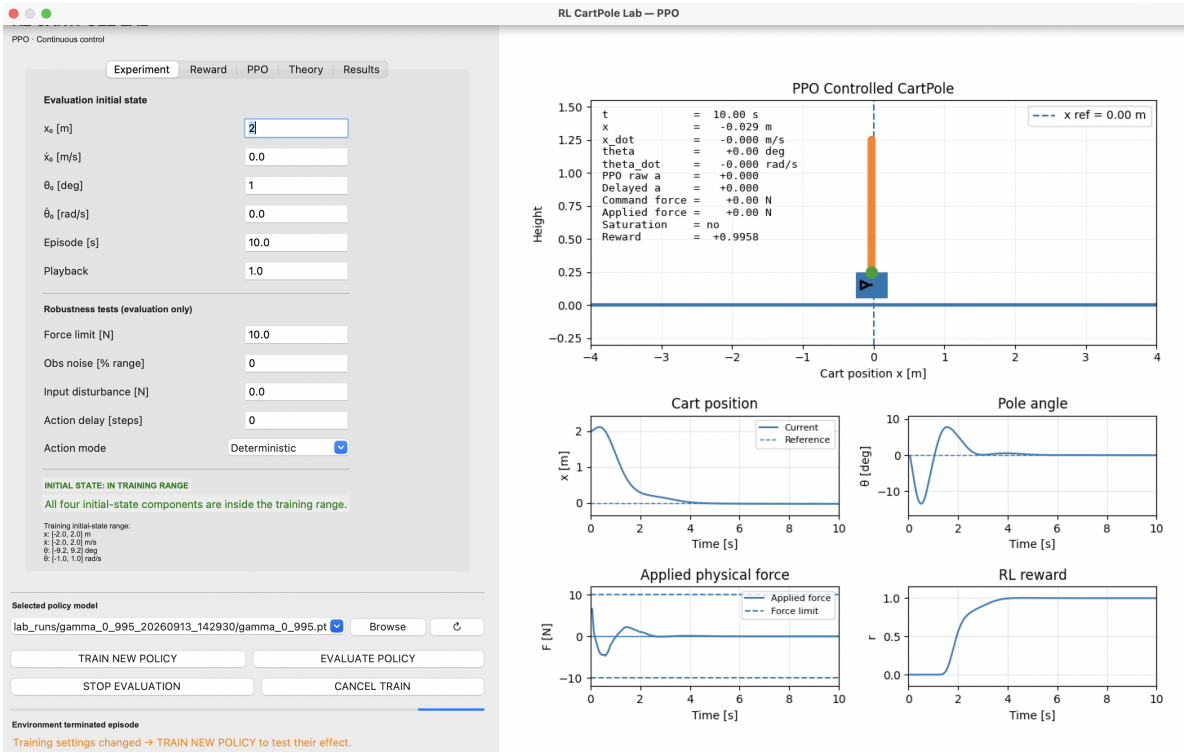
Policy'nin nominal koşullar dışındaki davranışını incelemek için Observation Noise, External Disturbance ve Action Delay seçenekleri ekledim.

Her Evaluation sonunda Final Error, RMSE, RMS Force, maximum Pole Angle, maximum Force, cumulative Reward ve Settling Time gibi performance metrics'i otomatik olarak hesapladım.

Farklı deneylerde train ettiğim Policy'leri daha kolay ayırabilmek için Policy'lere isim verme ve daha sonra bu modelleri GUI üzerinden seçme özelliği ekledim. Ayrıca iki farklı Evaluation sonucunu A ve B olarak kaydedip karşılaştırabileceğim bir yapı oluşturdum.

Son olarak State, Policy, Return, Value Function, TD Error, Advantage, GAE ve PPO clipped objective gibi çalışırken karşıma çıkan temel RL denklemlerini GUI içerisinde görebilmek için bir Theory paneli ekledim.

Bu şekilde hazır CartPole/PPO örneğini yalnızca çalıştırmak yerine, RL parametrelerini değiştirip sonuçlarını gözlemleyebileceğim ve farklı Policy'leri karşılaştırabileceğim küçük bir RL laboratuvarına dönüştürdüm.



Şekil 2: PPO training, evaluation, visualization ve metric extraction için geliştirdiğim RL CartPole Lab arayüzü.

3.3 Reward Design

CartPole için stage cost:

$$J_t = q_x(x_t - x_{\text{ref}})^2 + q_{\dot{x}}\dot{x}_t^2 + q_\theta(\theta_t - \theta_{\text{ref}})^2 + q_{\dot{\theta}}\dot{\theta}_t^2 + R_u u_t^2. \tag{33}$$

Exponential Reward mode için

$$r_t = \exp(-J_t) \tag{34}$$

kullandım.

3.4 LQR ile Bağlantı

Denklem (33) içerisindeki Reward yapısını, LQR'da kullanılan quadratic cost ile ilişkilendirdim:

$$J_{\text{LQR}} = \sum_{k=0}^{\infty} \left(\boldsymbol{x}_k^T Q \boldsymbol{x}_k + \boldsymbol{u}_k^T R \boldsymbol{u}_k \right). \tag{35}$$

Linear sistem için

$$\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k \tag{36}$$

tanımı altında LQR,

$$\boldsymbol{u}_k = -K\boldsymbol{x}_k \tag{37}$$

şeklinde linear State Feedback üretir.

Temel fark, PPO'nun nonlinear bir Policy'yi data/experience üzerinden öğrenmesi; LQR'ın ise linear model ve quadratic cost varsayımları altında analitik optimal feedback gain üretmesidir.

4 Evaluation Metrics

Tüm deneylerde aynı performance metrics'i kullandım.

Position RMSE'yi

$$\mathrm{RMSE}_x = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(x_k - x_{\mathrm{ref}})^2} \tag{38}$$

ve Pole Angle RMSE'yi

$$\mathrm{RMSE}_\theta = \sqrt{\frac{1}{N}\sum_{k=1}^{N}(\theta_k - \theta_{\mathrm{ref}})^2} \tag{39}$$

olarak hesapladım.

RMS Control Effort için

$$F_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}F_k^2} \tag{40}$$

ifadesini kullandım.

Bunlara ek olarak Final Position Error, maximum absolute Pole Angle, maximum absolute Force, cumulative episode Reward ve Settling Time değerlerine baktım.

5 Deneyler ve Sonuçlar

5.1 Deney 1: Initial-State Generalization

Bu deneyde aynı frozen Policy'nin farklı Initial State'lerden sistemi stabilize edip edemediğini inceledim. Deneyi Silver Lecture 1 ve Lecture 2'deki State-dependent Policy ve MDP kavramlarıyla ilişkilendirdim.

Kullandığım Initial State'ler:

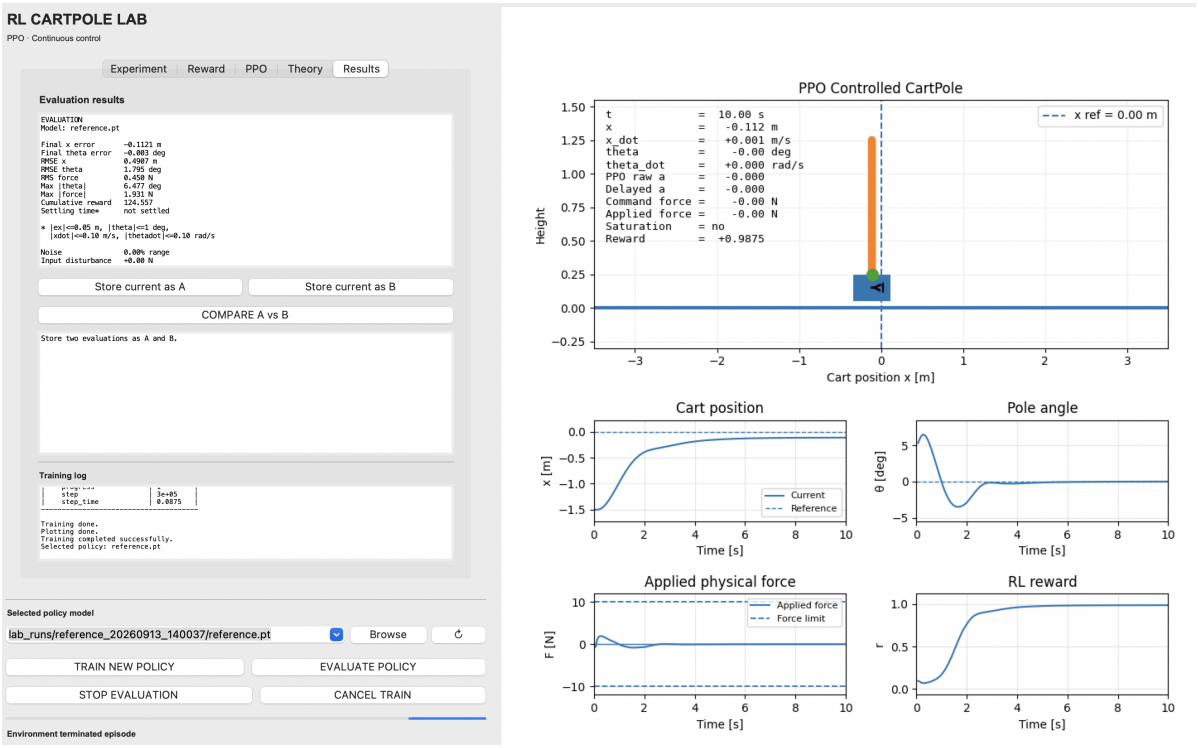
$$\boldsymbol{s}_0^{(1)} = [1,\ 0,\ 5^\circ,\ 0]^T, \tag{41}$$

$$\boldsymbol{s}_0^{(2)} = [-1,\ 0,\ -5^\circ,\ 0]^T, \tag{42}$$

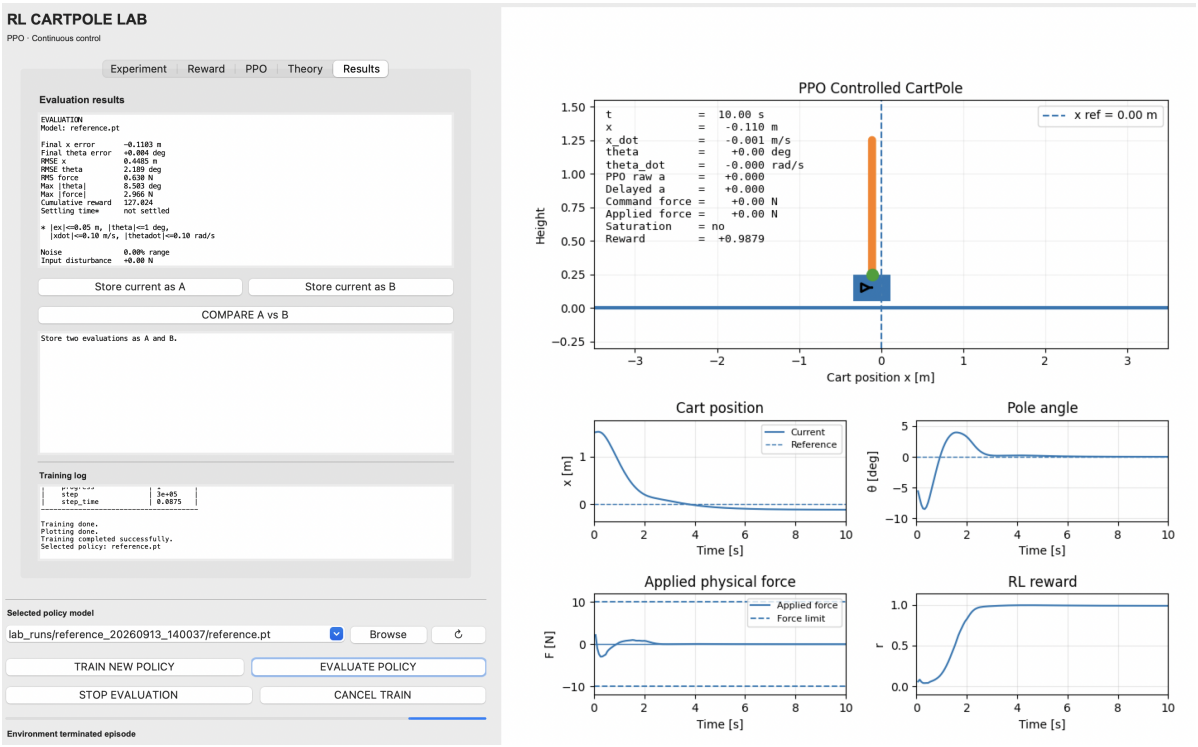
$$\boldsymbol{s}_0^{(3)} = [2,\ 0,\ 8^\circ,\ 0]^T. \tag{43}$$

Tablo 1: Aynı frozen Policy için Initial-State Generalization sonuçları.

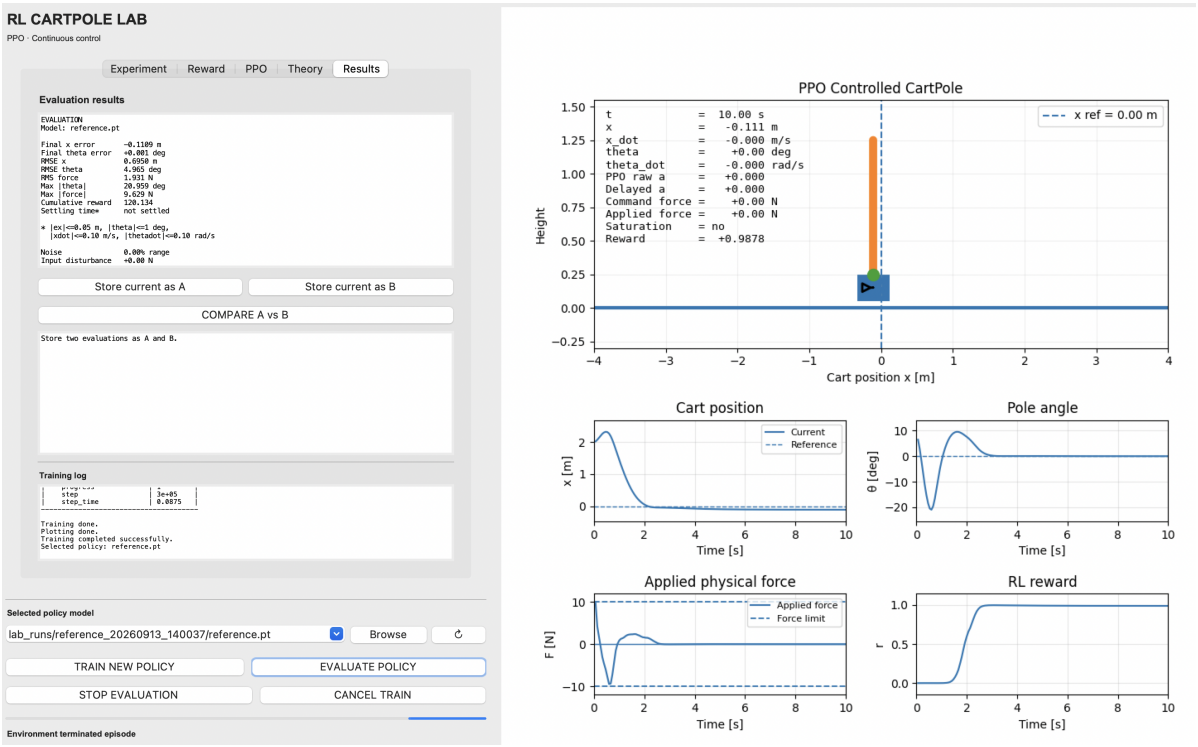
Test	Final x error [m]	RMSE$_x$ [m]	RMSE$_\theta$ [°]	F_{RMS} [N]	max $\lvert\theta\rvert$ [°]	max $\lvert F\rvert$ [N]	Return
$s_0^{(1)}$	-0.1121	0.4907	1.795	0.450	6.477	1.931	124.557
$s_0^{(2)}$	-0.1103	0.4485	2.189	0.630	8.503	2.966	127.024
$s_0^{(3)}$	-0.1109	0.6950	4.965	1.931	20.959	9.629	120.134



Şekil 3: Initial State: $s_0^{(1)} = [1,\ 0,\ 5°,\ 0]^T$ için CartPole response'u.



Şekil 4: Initial State: $s_0^{(2)} = [-1,\ 0,\ -5°,\ 0]^T$ için CartPole response'u.



Şekil 5: Initial State: $s_0^{(3)} = [2,\ 0,\ 8°,\ 0]^T$ için CartPole response'u.

Aynı Policy'nin üç farklı Initial State için Pole Angle'ı stabilize ettiğini ve velocity bileşenlerini sıfıra yaklaştırdığını gördüm. Initial condition zorlaştıkça gereken Control Effort da arttı. En son durumda Force actuator sat limitine yaklaştı.

Üç response'un da yaklaşık aynı cart position'a yakınsadı:

$$x_{\text{ss}} \approx -0.11 \text{ m}. \tag{44}$$

RL açısından bu sonucu, Actor'ın tek bir trajectory ezberlemek yerine sistemin anlık State'ine göre farklı Action üreten bir feedback Policy öğrendiği şeklinde yorumladım. LQR da $u = -Kx$ şeklinde State Feedback üretir; ancak LQR mapping'i model tabanlı ve linear iken PPO Policy nonlinear ve experience üzerinden öğrenilir.

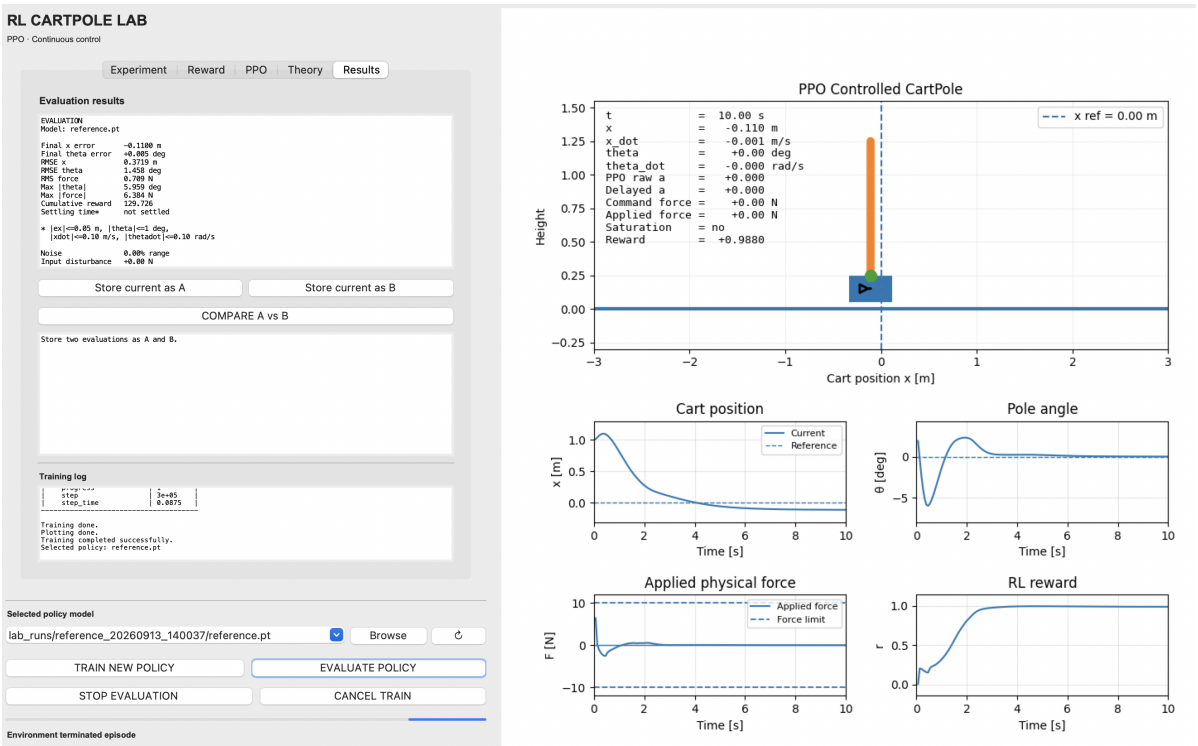
5.2 Deney 2: Position Reward Weight q_x Etkisi

Bu deneyde Denklem (33) içerisindeki yalnızca cart-position weight q_x değerini değiştirerek Reward Shaping etkisini inceledim:

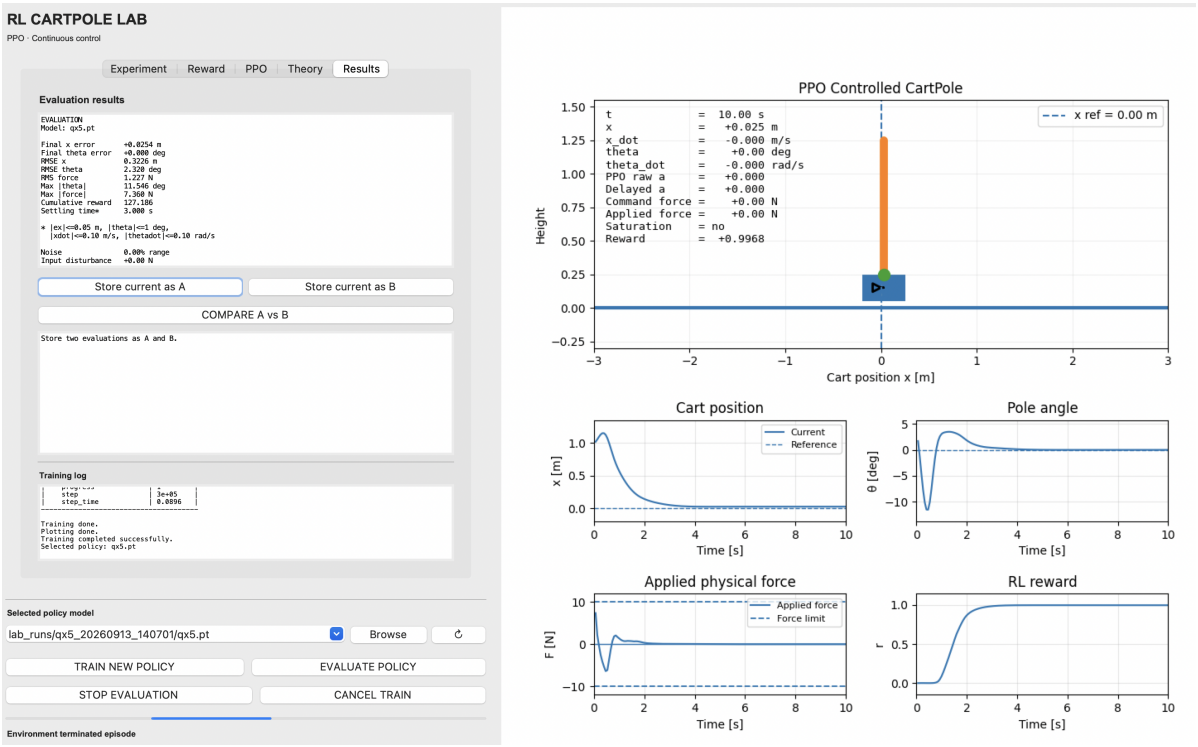
$$q_x \in \{1,\ 5,\ 10\}. \tag{45}$$

Tablo 2: Position Reward weight q_x değişiminin etkisi.

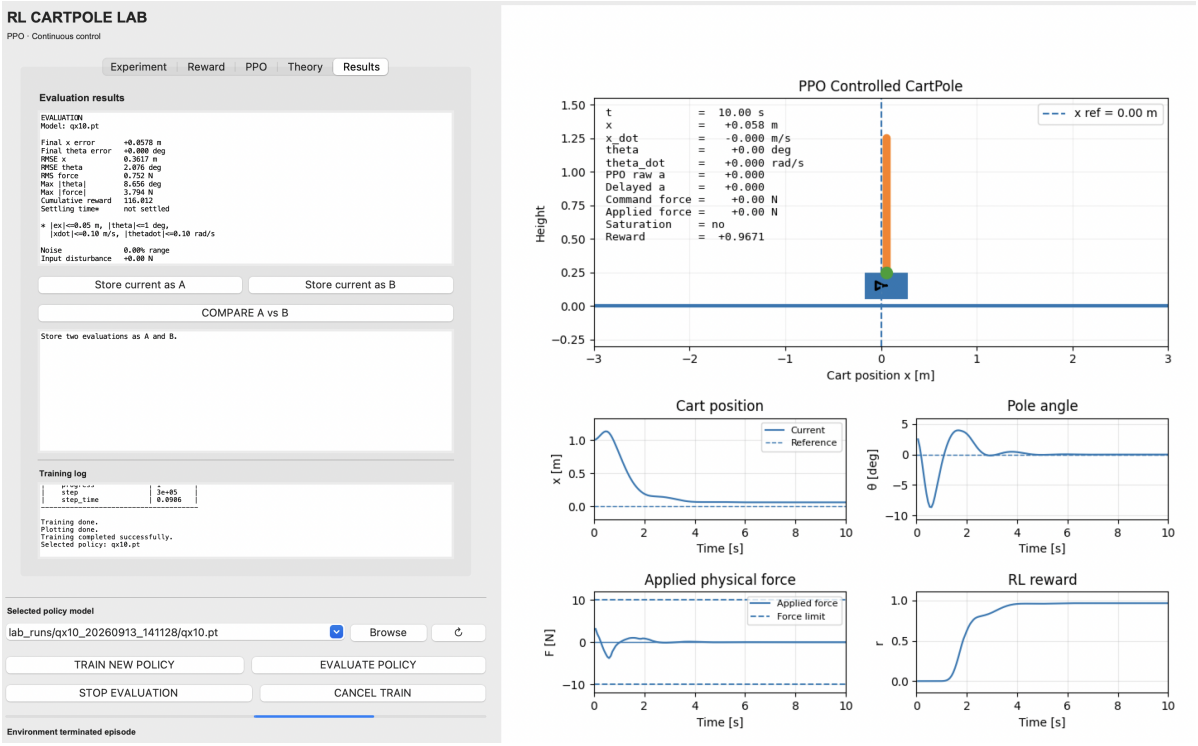
q_x	Final x error [m]	RMSE$_x$ [m]	RMSE$_\theta$ [°]	F_{RMS} [N]	max $\lvert\theta\rvert$ [°]	max $\lvert F\rvert$ [N]	Settling
1	-0.1100	0.3719	1.458	0.709	5.959	6.384	Not settled
5	+0.0254	0.3226	2.320	1.227	11.546	7.360	3.000 s
10	+0.0578	0.3617	2.076	0.752	8.656	3.794	Not settled



Şekil 6: $q_x = 1$ için trained Policy response'u.



Şekil 7: $q_x = 5$ için trained Policy response'u.



Şekil 8: $q_x = 10$ için trained Policy response'u.

q_x değerini 1'den 5'e çıkardığımda steady-state position error'ın belirgin şekilde azaldığını ve RMSE_x değerinin düştüğünü gördüm. Buna karşılık Control Effort ve transient Pole Angle excursion arttı.

q_x değerini 10'a çıkardığımda ise $q_x = 5$ durumuna göre ek bir improvement görmedim. PPO training stochastic, nonlinear ve approximate bir optimization süreci olduğu için bu sonucu beklenebilir

buldum.

Test ettiğim ayarlar içerisinde $q_x = 5$ Policy'si en iyi position regulation sonucunu verdi ve tanımladığım Settling criterion'u sağlayan tek Policy oldu.

LQR bağlantısı. q_x artırılmasını, LQR cost içerisindeki Q matrisinde position state'e karşılık gelen diagonal weight'in artırılmasına benzettim. Her iki durumda da position error diğer objective'lere göre daha pahalı hale geliyor. Fakat LQR yeni bir optimal linear gain hesaplarken PPO yeni Policy'yi interaction yoluyla yeniden öğreniyor.

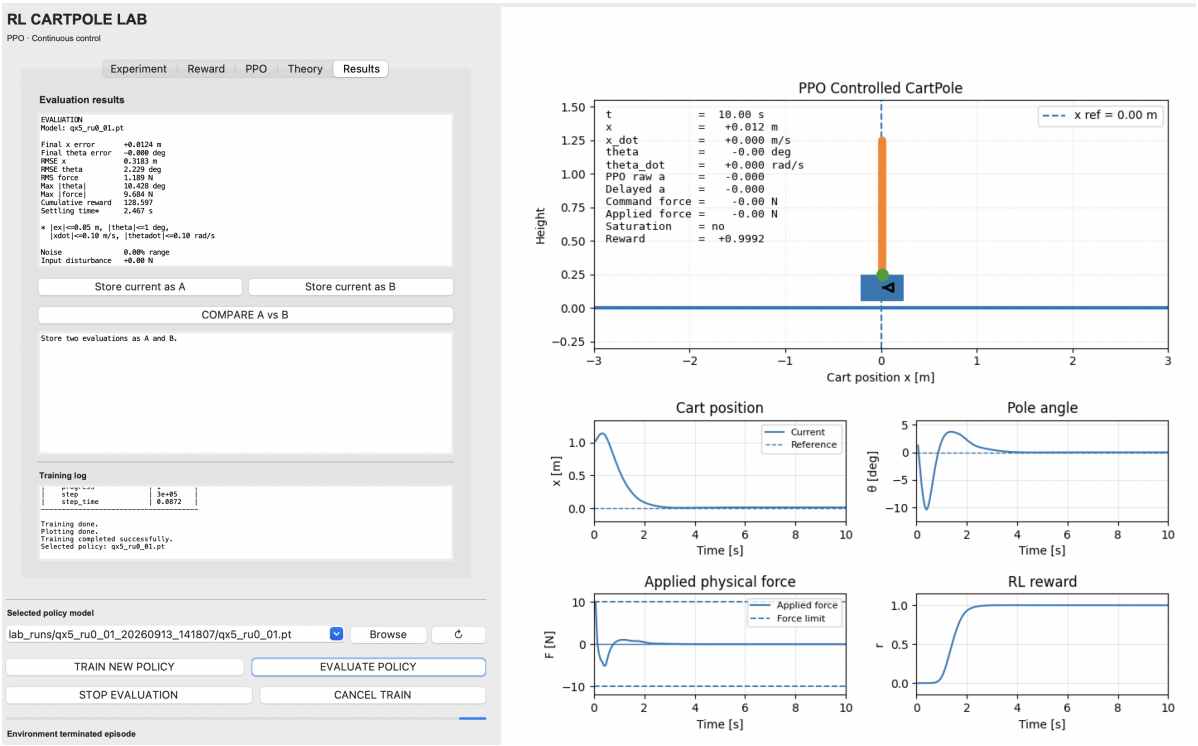
5.3 Deney 3: Control-Effort Penalty R_u Etkisi

Bu deneyde regulation performance ile Control Effort arasındaki trade-off'u inceledim. $q_x = 5$ değerini sabit tutup iki farklı Control-Effort penalty değeri kullandım:

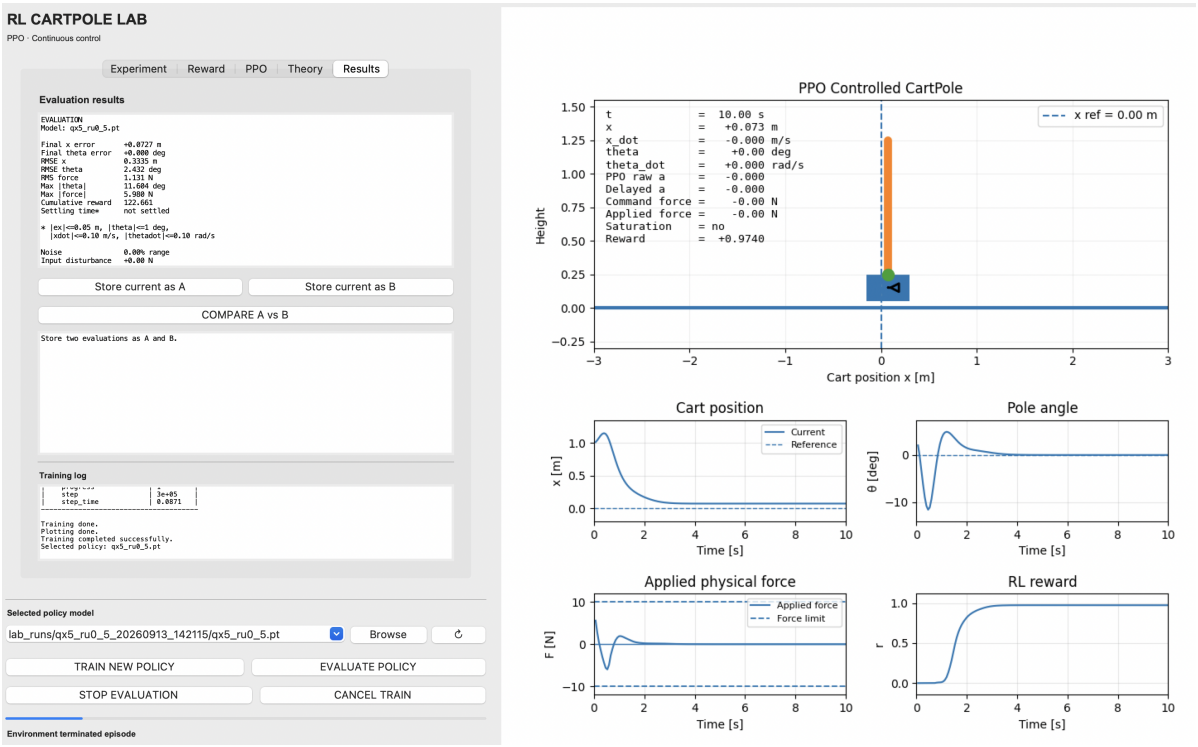
$$R_u \in \{0.01,\ 0.50\}. \tag{46}$$

Tablo 3: $q_x = 5$ için Control-Effort penalty R_u etkisi.

R_u	Final x error [m]	RMSE$_x$ [m]	RMSE$_\theta$ [°]	F_{RMS} [N]	max $\lvert\theta\rvert$ [°]	max $\lvert F\rvert$ [N]	Settling
0.01	+0.0124	0.3183	2.229	1.189	10.428	9.684	2.467 s
0.50	+0.0727	0.3335	2.432	1.131	11.604	5.980	Not settled



Şekil 9: $R_u = 0.01$ için trained Policy response'u.



Şekil 10: $R_u = 0.50$ için trained Policy response'u.

R_u değerini 0.01'den 0.50'ye çıkardığımda peak Force'un

$$9.684 \text{ N} \rightarrow 5.980 \text{ N} \tag{47}$$

şeklinde azaldığını gördüm. Buna karşılık Final Position Error

$$0.0124 \text{ m} \rightarrow 0.0727 \text{ m} \tag{48}$$

şeklinde arttı. Ayrıca $R_u = 0.01$ durumunda sistem 2.467 s içerisinde Settling criterion'u sağlarken, $R_u = 0.50$ durumunda test süresi boyunca Settling criterion sağlanmadı.

Bu sonuçtan, Control Effort'u Reward içerisinde daha fazla cezalandırdığımda Policy'nin daha düşük peak Force kullanmayı öğrendiğini, ancak bunun position regulation performance'ında bir kayıp oluşturduğunu gözlemledim. Test ettiğim iki durum için temel trade-off'u

$$R_u \uparrow \quad \Rightarrow \quad \max|F| \downarrow, \qquad |e_{x,\text{ss}}| \uparrow \tag{49}$$

şeklinde özetledim.

RMS Force değerinde de 1.189 N'dan 1.131 N'a küçük bir azalma gördüm. Ancak Control-Effort penalty etkisi peak Force üzerinde çok daha belirgin olduğu için karşılaştırmada özellikle maximum Force ve Settling Time değerlerine odaklandım.

LQR bağlantısı. Bu deneyi LQR'daki R weight değişimiyle ilişkilendirdim. LQR'da R değeri arttıkça Control Input kullanımı daha pahalı hale gelir ve controller genellikle daha az agresif davranır. PPO'da da benzer şekilde R_u değerini artırdığımda Policy daha düşük Force kullanmayı öğrendi; ancak bunun karşılığında position tracking/regulation performance'ı kötüleşti.

5.4 Deney 4: Discount Factor γ Etkisi

Discount Factor'ın immediate ve future Reward'ların göreli önemini nasıl değiştirdiğini incelemek için Discounted Return ifadesini

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{50}$$

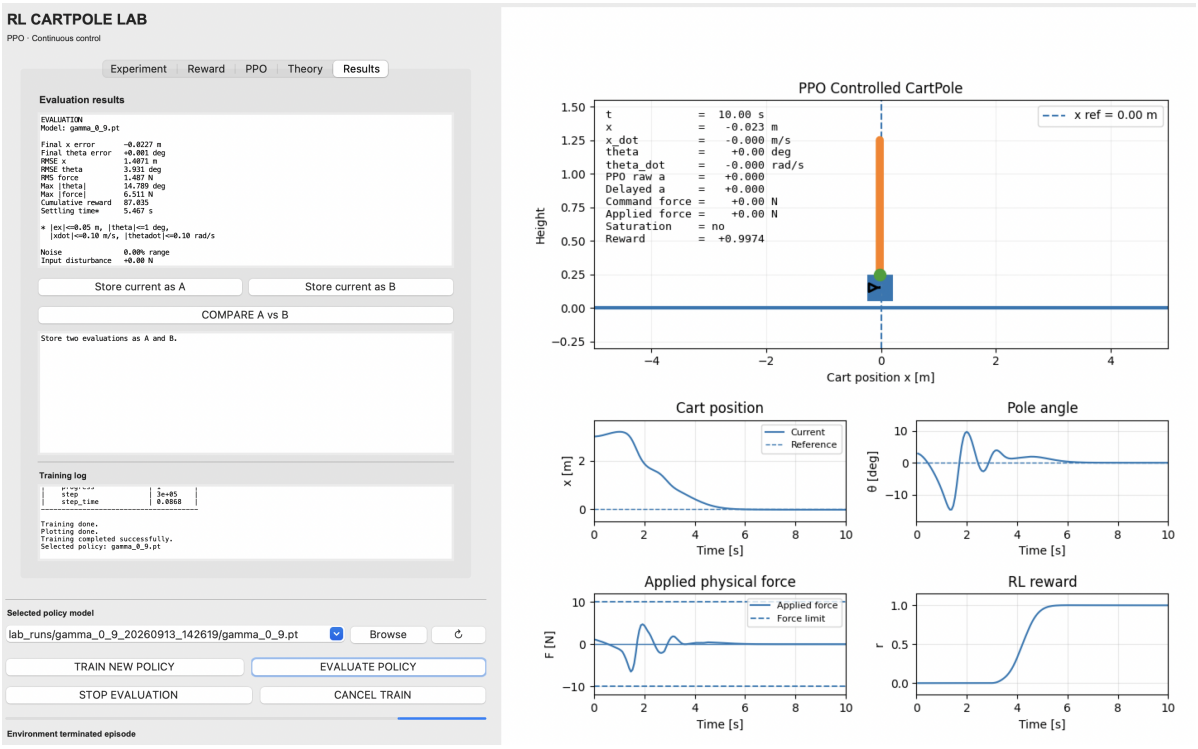
kullandım ve bu denklemdeki γ parametresini değiştirdim.

$$\gamma \in \{0.90,\ 0.995\} \tag{51}$$

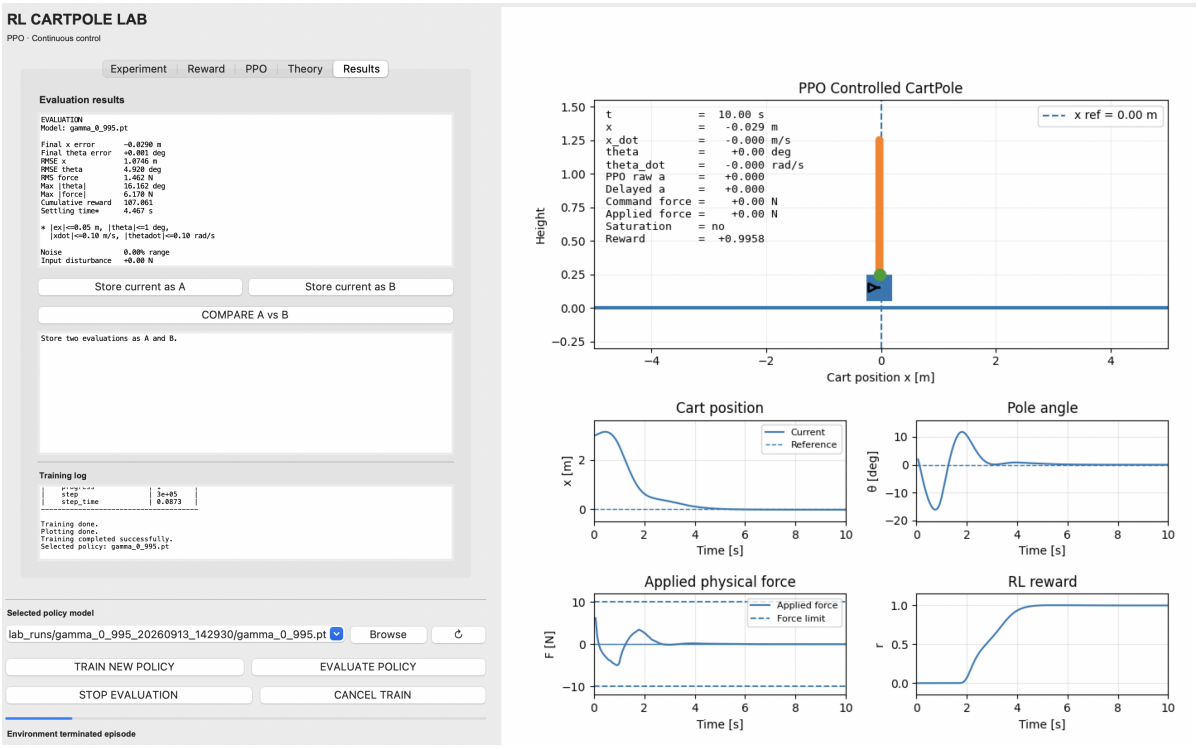
değerleriyle iki ayrı Policy train ettim.

Tablo 4: Discount Factor γ değişiminin etkisi. İki durumda Reward Function aynı olduğu için cumulative Reward değerlerini karşılaştırdım.

γ	Final x error [m]	RMSE$_x$ [m]	RMSE$_\theta$ [°]	F_{RMS} [N]	max $\lvert\theta\rvert$ [°]	Return	Settling
0.900	-0.0227	1.4071	3.931	1.487	14.789	87.035	5.467 s
0.995	-0.0290	1.0746	4.920	1.462	16.162	107.061	4.467 s



(a) $\gamma = 0.90$ için trained Policy response'u.



(b) $\gamma = 0.995$ için trained Policy response'u.

Şekil 11: $\gamma = 0.90$ ve $\gamma = 0.995$ ile trained Policy karşılaştırması.

$\gamma = 0.995$ ile train ettiğim Policy'nin daha düşük cart-position RMSE, daha kısa Settling Time ve daha yüksek cumulative Reward elde ettiğini gördüm. Buna karşılık Pole Angle RMSE ve maximum angular excursion daha büyük çıktı.

Bu sonucu, daha yüksek Discount Factor kullanılan Policy'nin short-term angular deviation'ı daha

fazla kabul edip long-term position regulation açısından daha iyi bir sonuç elde etmesi şeklinde yorumladım.

Bu davranışı Silver Lecture 2'deki Discounted Return tanımıyla ilişkilendirdim. Örneğin

$$0.9^{50} \approx 0.005, \qquad 0.995^{50} \approx 0.778. \tag{52}$$

Bu nedenle $\gamma = 0.995$ durumunda uzak gelecekteki Reward'ların bugünkü decision üzerindeki etkisi çok daha güçlü.

Buradan "daha büyük γ her zaman daha iyidir" sonucunu çıkarmadım. Daha uzun effective horizon, Value estimation ve optimization sürecini de zorlaştırabilir.

5.5 Deney 5: Observation-Noise Robustness

Son deneyde aynı frozen Policy'yi nominal ve noisy State observation altında evaluate ettim. Yeniden training yapmadım. Policy'ye gerçek State yerine

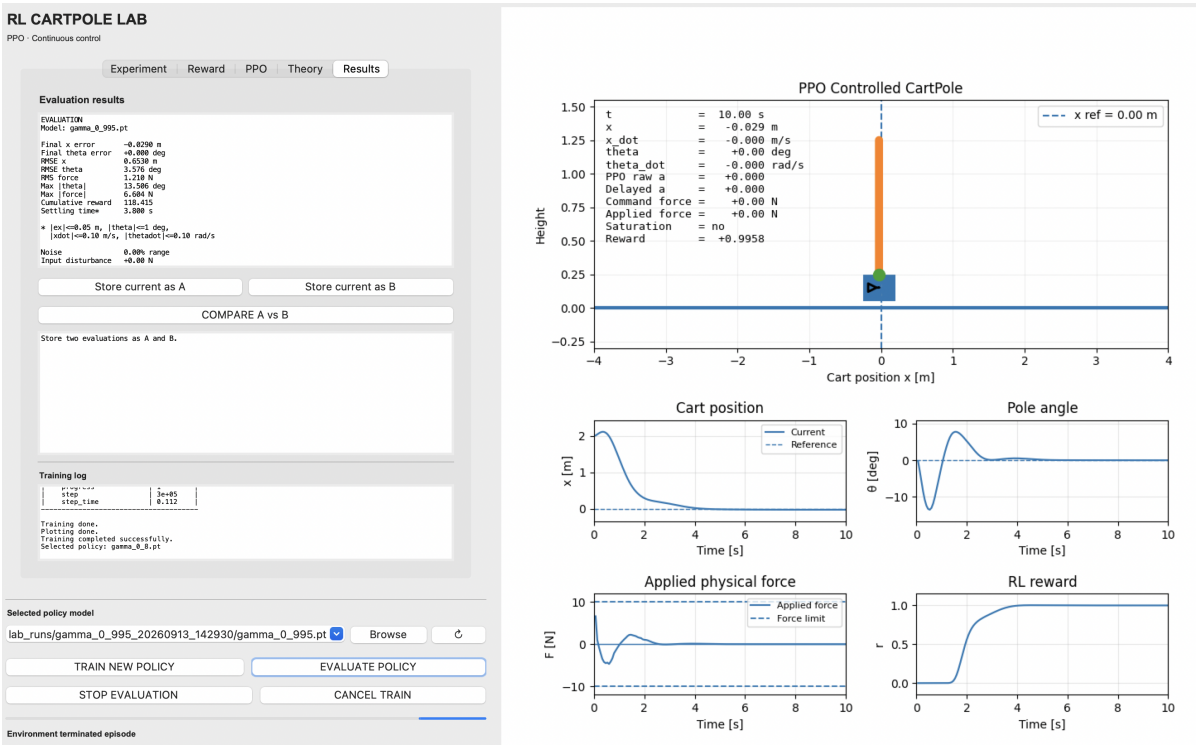
$$\tilde{\boldsymbol{s}}_t = \boldsymbol{s}_t + \boldsymbol{n}_t \tag{53}$$

verdim.

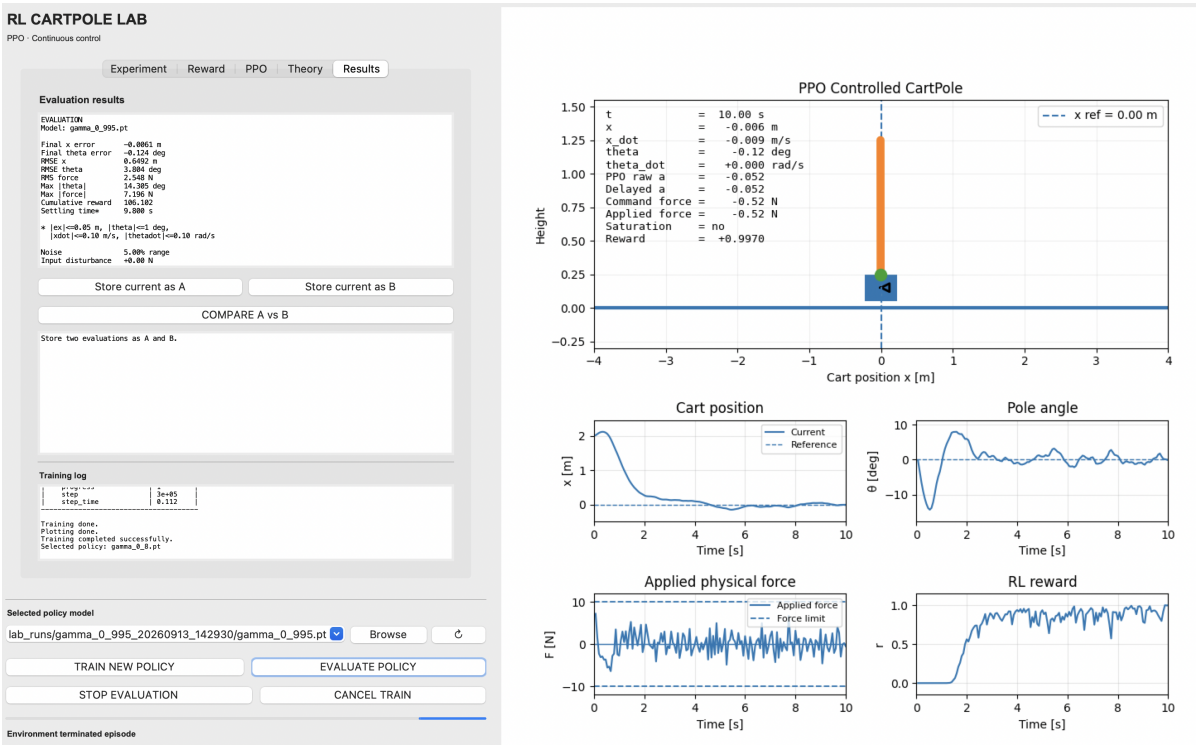
0% ve 5% Observation Noise durumlarını karşılaştırdım.

Tablo 5: Aynı frozen Policy'nin Observation Noise altında robustness sonuçları.

Noise	Final x error [m]	RMSE_x [m]	RMSE_θ [°]	F_{RMS} [N]	$\max\lvert\theta\rvert$ [°]	$\max\lvert F\rvert$ [N]	Settling
0%	-0.0290	0.6530	3.576	1.210	13.506	6.604	3.800 s
5%	-0.0061	0.6492	3.804	2.548	14.305	7.196	9.800 s



(a) 0% Observation Noise için evaluation response'u.



(b) 5% Observation Noise için evaluation response'u.

Şekil 12: Aynı frozen PPO Policy için 0% ve 5% Observation Noise karşılaştırması.

RMS Force

$$1.210 \text{ N} \quad \text{değerinden} \quad 2.548 \text{ N} \tag{54}$$

değerine çıkarken Settling Time

$$3.8 \text{ s} \quad \text{değerinden} \quad 9.8 \text{ s} \tag{55}$$

değerine çıktı.

Noisy observation nedeniyle Policy'nin equilibrium yakınında bile measurement fluctuation'lara tepki vermeye devam ettiğini gördüm. Bunun sonucunda Action jitter, küçük State oscillation'ları ve Reward fluctuation oluştu.

David Silver Lecture 1'deki Fully Observable ve Partially Observable Environment kısmı önemliydi. Nominal CartPole setup'ta Agent complete State'e erişirken, Observation Noise eklediğimde Policy artık Environment State'ini tam olarak gözlemleyemedi.